COL774: Machine Learning
Assignment 3

Ankur Kumar (2025AIB2557)
Indian Institute of Technology Delhi

1. Decision Tree and Random Forest

1.1. Decision Tree Construction

In this section, we constructed a decision tree classifier and evaluated its performance using different maximum depths. Table 1 summarizes the training, validation, and test accuracies for each depth, while Figure 1 provides a visual comparison of these accuracies.

As expected, the training accuracy increases with tree depth, because deeper trees can fit the training data more precisely. However, this improved fit does not always translate to better generalization. The validation and test accuracies reveal that the model does not generalize well at greater depths. Specifically, we observe a slight improvement in validation and test accuracies around depth 3, but beyond that, the performance decreases or remains roughly constant. This behavior is indicative of overfitting, where the model captures noise in the training data rather than underlying patterns.

Depth	Train Accuracy	Validation Accuracy	Test Accuracy
1	0.575	0.560	0.558
3	0.722	0.600	0.613
5	0.884	0.592	0.594
8	0.985	0.590	0.593
10	0.995	0.587	0.588
15	0.995	0.587	0.588
20	0.995	0.587	0.588
25	0.995	0.587	0.588

Table 1: Train, validation, and test accuracies for decision trees with varying maximum depths.

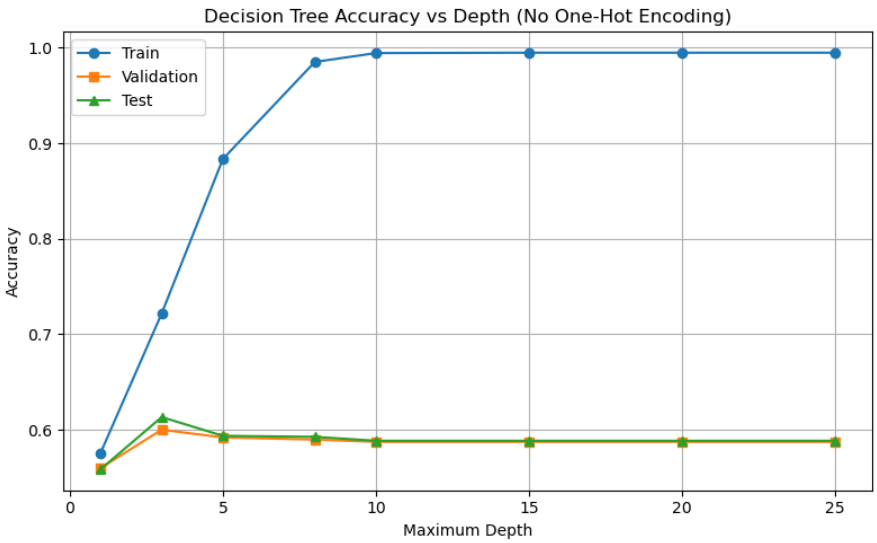

Figure 1: Training, Validation, and Test Accuracies vs Maximum Depth

1.2. Decision Tree with one hot encoding

With one-hot encoding applied to categorical features having more than two categories, we observe that the training accuracy increases **more gradually** with tree depth. This behavior occurs because one-hot encoding expands the feature space, requiring **deeper trees** to achieve classification accuracy comparable to the original dataset.

Table 2 summarizes the training, validation, and test accuracies for decision trees trained with and without one-hot encoding. Figure 2 visualizes these accuracies, allowing for a direct comparison. It can be seen that while training accuracy improves steadily with depth in the one-hot encoded dataset, the validation and test accuracies show more moderate improvements, indicating that the expanded feature space helps the model capture categorical information more effectively but still risks overfitting at higher depths.

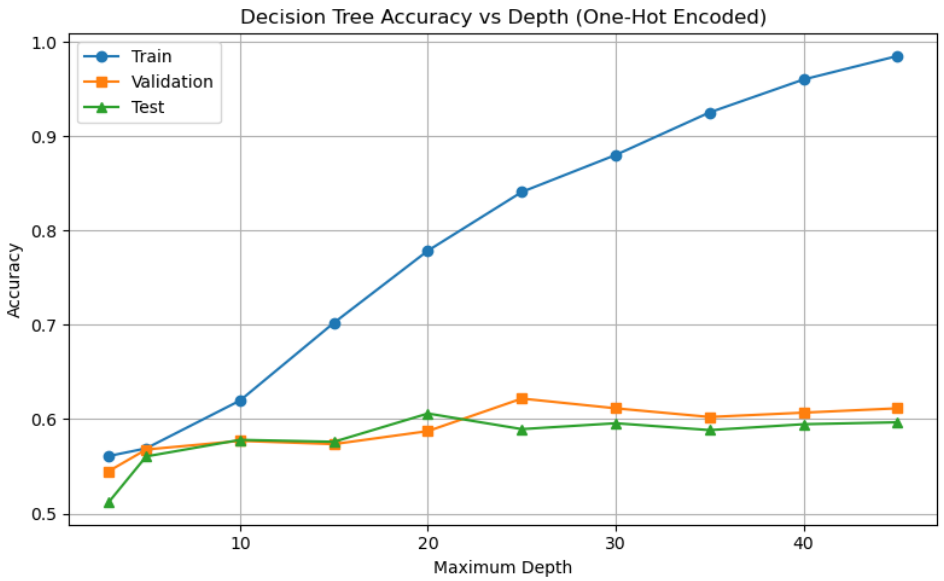

Figure 2: Training, Validation, and Test Accuracies for Decision Trees with One-Hot Encoding.

Depth	Without One-Hot Encoding			With One-Hot Encoding		
	TrainAcc	ValAcc	TestAcc	TrainAcc	ValAcc	TestAcc
3	0.722	0.600	0.614	0.561	0.545	0.512
5	0.878	0.582	0.597	0.569	0.568	0.560
10	0.994	0.578	0.590	0.620	0.577	0.578
15	0.994	0.578	0.590	0.702	0.574	0.576
20	0.994	0.578	0.590	0.779	0.587	0.606
25	0.994	0.578	0.590	0.841	0.622	0.589
30	0.994	0.578	0.590	0.880	0.611	0.596
35	0.994	0.578	0.590	0.925	0.602	0.588
40	0.994	0.578	0.590	0.960	0.607	0.595
45	0.994	0.578	0.590	0.985	0.611	0.597

Table 2: Comparison of Decision Tree Accuracies With and Without One-Hot Encoding

1.3. Decision Tree Post Pruning

In this experiment, **post-pruning** was applied to decision trees trained at various maximum depths (10–45) to mitigate overfitting and improve generalization. After training a fully grown decision tree, pruning was performed iteratively using the validation set as a guide. At each step, every non-leaf node was tentatively replaced by a leaf node labeled with the majority class of its descendants, and the change was retained only if it **increased the validation accuracy**. If validation accuracy decreased, the original subtree was restored. This process continued until no further improvement was observed.

As shown in Table 3, pruning substantially reduced the number of nodes, simplifying the trees while maintaining or improving validation performance. Notably, the validation accuracy generally **increased with depth up to around 30**, after which gains stabilized or slightly declined, suggesting an optimal trade-off between

model complexity and generalization. Training accuracy remained moderate, while test accuracy followed the same upward trend as validation accuracy, confirming that pruning effectively reduced overfitting.

The corresponding accuracy trends are visualized in Figure 3. The curves show that as pruning reduces tree size, validation accuracy initially improves—indicating successful removal of overfitted subtrees—before plateauing. Overall, pruning helped the models generalize better by removing unnecessary complexity.

Max Depth	Nodes Before	Nodes After	Train Accuracy	Validation Accuracy	Test Accuracy
10	433	51	0.5983	0.5908	0.5719
15	1671	107	0.5949	0.6046	0.5491
20	2885	137	0.6089	0.6138	0.5926
25	3429	175	0.6282	0.6506	0.5812
30	3889	187	0.6319	0.6575	0.5988
35	4463	163	0.6249	0.6483	0.5884
45	5425	161	0.6211	0.6402	0.5832

Table 3: Performance of Decision Trees Before and After Post-Pruning at Different Maximum Depths

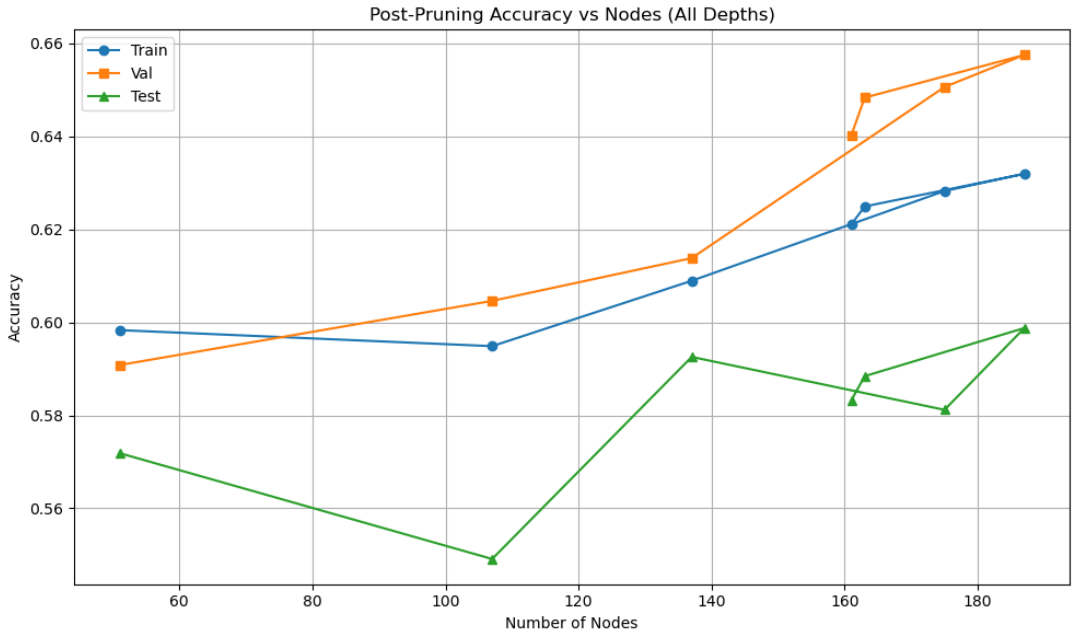

Figure 3: Post-pruning effect on training, validation, and test accuracies across different tree sizes.

1.4. Pruning with Gini Impurity

In this part, the decision tree construction and pruning were repeated using **Gini Impurity** as the splitting criterion instead of Information Gain (Entropy). The Gini criterion is another popular measure of node impurity and is used as the default option in several machine learning libraries such as `scikit-learn` due to its computational efficiency.

The **Gini Impurity** for a node is defined as:

$$Gini(t) = 1 - \sum_{i=1}^{C} p_i^2$$

where p_i is the proportion of samples belonging to class i in that node, and C is the total number of classes. A perfectly pure node (all samples from one class) has $Gini = 0$, while higher values indicate greater class mixing or impurity.

For a given split, the reduction in impurity (analogous to information gain) is computed as:

$$\Delta Gini = Gini(parent) - \sum_{k=1}^{K} \frac{N_k}{N} \times Gini(child_k)$$

where N_k is the number of samples in the k^{th} child node and N is the total number of samples in the parent node. The split with the largest reduction in impurity is selected.

Compared to the entropy criterion, Gini impurity avoids logarithmic computations and is therefore slightly faster to compute. In large datasets with many features, this computational advantage can become significant. Empirically, both criteria often produce very similar trees. However, Gini impurity tends to create splits that isolate the majority class in a node more aggressively, whereas entropy may lead to slightly more balanced partitions.

The same post-pruning (reduced-error pruning) strategy was applied to trees trained using Gini impurity. The pruning process was guided by validation accuracy, and the tree was simplified iteratively until no further improvement was observed. The validation accuracy after pruning generally increased, confirming that pruning successfully reduced overfitting and improved generalization.

Overall, both entropy and Gini impurity produced comparable results on this dataset, but Gini achieved similar performance with a marginally simpler tree structure and faster computation. The detailed comparison of validation and test accuracies for gini criteria at different depth is shown in Table 4, and their corresponding performance trends are illustrated in Figure 4.

Max Depth	Nodes Before Pruning	Nodes After Pruning	Validation Acc. (Before)	Validation Acc. (After)	Test Acc. (After)
10	447	73	0.6011	0.6195	0.5667
15	1819	81	0.5977	0.6276	0.5822
20	3357	91	0.5943	0.6402	0.5729
25	4263	165	0.5828	0.6402	0.5863
30	4873	139	0.5793	0.6379	0.5812
35	5159	179	0.5839	0.6506	0.5853
45	5629	137	0.5782	0.6356	0.5791

Table 4: Performance of Gini Impurity Based Decision Tree at Different Depths (After Pruning)

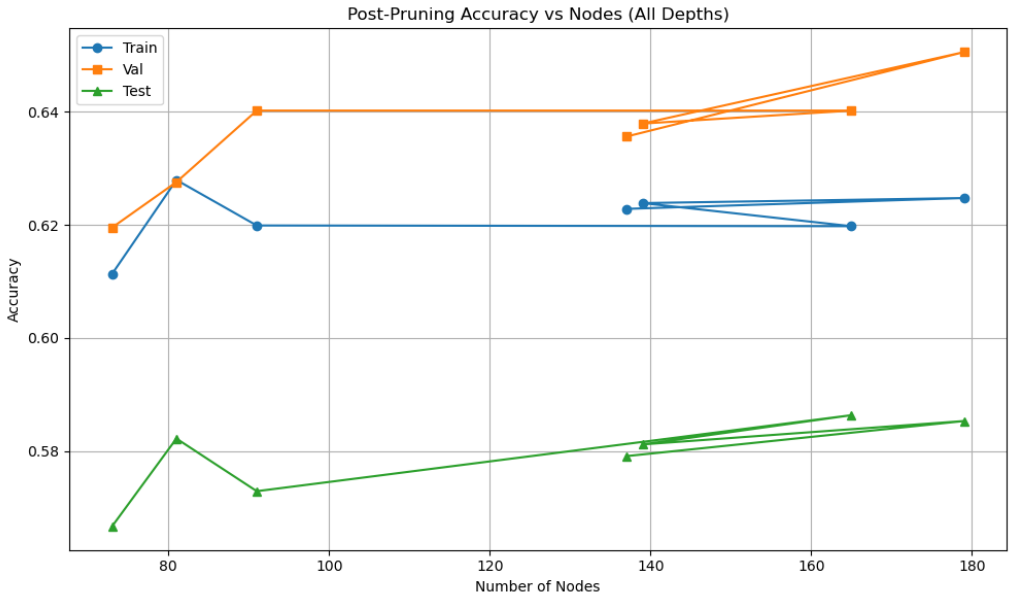

Figure 4: Comparison of Validation and Test Accuracy for Gini Impurity Based Trees

1.5. Decision Tree scikit learn

1.5.1. Varying `max_depth`

In this experiment, we varied the maximum depth of the decision tree to study its effect on model performance. The results are summarized in Table 5.

Table 5: Accuracy for varying `max_depth`

Max Depth	Train Accuracy	Validation Accuracy	Test Accuracy
15	0.826	0.617	0.634
25	0.985	0.609	0.622
35	0.999	0.611	0.620
45	1.000	0.623	0.618

The best validation accuracy was obtained at a maximum depth of 45, giving a validation accuracy of 0.623. We can observe that training accuracy increases sharply with depth, eventually reaching 1.0, which indicates **overfitting**. The validation and test accuracies, however, saturate and even start to slightly degrade.
Observation: In our own implementation, the results were generally lower compared to `scikit-learn`'s optimized Decision Tree model. This difference arises mainly because our implementation:

- considers only a single split point (median) per feature rather than all possible thresholds,
- lacks cost-complexity pruning and other regularization strategies,
- and uses a less optimized impurity computation.

These factors limit the ability of our model to capture optimal decision boundaries, leading to slightly poorer generalization performance.

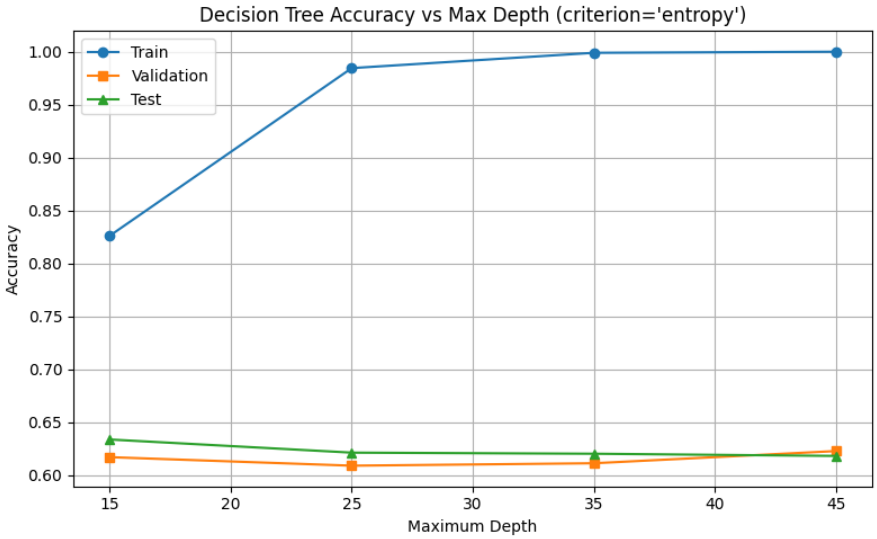

Figure 5: Effect of varying `max_depth` on training, validation, and test accuracy.

—

1.5.2. Varying `ccp_alpha`

Next, we varied the pruning parameter `ccp_alpha` to investigate its impact on the decision tree's complexity and generalization ability. The results are shown in Table 6.

Table 6: Accuracy for varying `ccp_alpha`

ccp_alpha	Train Accuracy	Validation Accuracy	Test Accuracy
0.0000	1.000	0.623	0.618
0.0001	1.000	0.623	0.618
0.0003	0.978	0.622	0.615
0.0005	0.864	0.641	0.612

The best validation accuracy was achieved at `ccp_alpha = 0.0005`, yielding a validation accuracy of 0.641. Increasing `ccp_alpha` prunes the tree more aggressively, which helps reduce overfitting but can also slightly lower the training accuracy.

Observation: Again, our custom implementation showed weaker performance compared to the built-in model, primarily because:

- pruning was overly simplistic,
- impurity computation and threshold selection were less fine-grained,
- and sklearn's model benefits from highly optimized C-based routines.

As a result, while both models show the same general trends (higher pruning → less overfitting), the sklearn implementation generalizes more effectively overall.

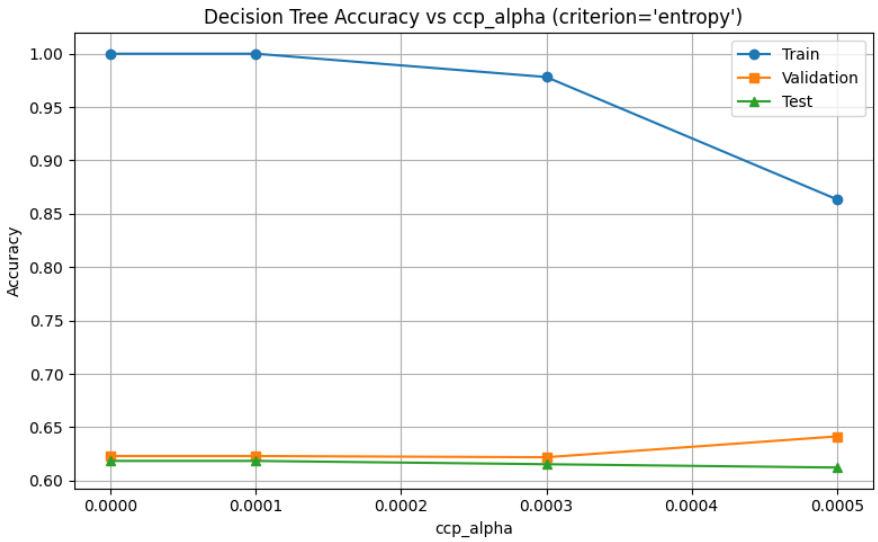

Figure 6: Effect of varying `ccp_alpha` on training, validation, and test accuracy.

1.6. Random Forest

Random Forests are an extension of decision trees in which multiple decision trees are grown in parallel on *bootstrapped samples* constructed from the original training data. Each tree is trained independently, and predictions are made by aggregating the outputs of all the trees (majority voting for classification). Random Forests help reduce overfitting that is common in single decision trees and improve generalization.

In this experiment, we used the `RandomForestClassifier` from `scikit-learn` with `criterion="entropy"` to be consistent with our decision tree experiments. We performed a *grid search* over the following parameters to find the best combination:

- **Number of estimators** (*n_estimators*): 50, 150, 250, 350
- **Maximum features** (*max_features*): 0.1, 0.3, 0.5, 0.7, 0.9
- **Minimum samples split** (*min_samples_split*): 2, 4, 6, 8, 10

For each configuration, the model was trained on the training set, and accuracies were evaluated on the training, out-of-bag (OOB), validation, and test sets. Categorical features were encoded using *label encoding* to convert string features into numeric labels, which is required for `scikit-learn` models.

The optimal hyperparameters and corresponding accuracies are reported in Table 7.

Table 7: Random Forest: Optimal Hyperparameters and Accuracies

n_estimators	max_features	min_samples_split	Train Acc.	OOB Acc.	Val Acc.	Test Acc.
150	0.1	6	0.983	0.718	0.718	0.701

Observations from the results:

- The Random Forest achieved very high training accuracy, close to 100%, which is expected since it aggregates multiple decision trees.

- The validation accuracy (0.718) is higher than the validation accuracy obtained in parts (c) and (d) with single decision trees, showing that Random Forests generalize better.
- The out-of-bag (OOB) score provides a good estimate of generalization without needing a separate validation set.
- The test accuracy (0.701) is slightly lower than validation, which is common due to variance between datasets, but overall Random Forest improves generalization compared to single decision trees, where overfitting was more prominent.

In conclusion, using Random Forests with a proper grid search for hyperparameters resulted in a model that achieves better validation performance and reduced overfitting compared to our own decision tree implementation.

2. Neural Networks

2.1. Code Implementation

The implementation of the neural network architecture for the specified configurations is available at this link. The file *neural_network.py* defines the *NeuralNet* class, which is utilized throughout this question for various experimental setups.

2.2. Single Hidden Layer

Stopping Criterion: The training process was allowed to run for a maximum of **300 epochs** for all network configurations. Additionally, an **early stopping** condition was applied — training was terminated if the change in loss between consecutive epochs fell below a small tolerance threshold. This combined stopping criterion (maximum epochs or minimal loss improvement) ensures both computational efficiency and stable convergence across models with different hidden layer sizes.

Performance Metrics: The table below reports the average precision, recall, and F1 score for both the training and test datasets at different hidden layer sizes. These metrics were computed using `scikit-learn`'s `precision_recall_fscore_support` utility.

Hidden Units	**Train Metrics**			**Test Metrics**		
	Precision	Recall	F1	Precision	Recall	F1
1	0.031	0.070	0.028	0.026	0.068	0.027
5	0.407	0.429	0.401	0.367	0.388	0.361
10	0.636	0.639	0.636	0.554	0.559	0.554
50	0.909	0.908	0.908	0.761	0.761	0.761
100	0.969	0.969	0.969	0.821	0.821	0.821

Table 8: Precision, Recall, and F1 Scores for Different Hidden Layer Sizes

F1 Score Visualization: The following figure illustrates how the average F1 score varies with the number of hidden units for both training and testing datasets.

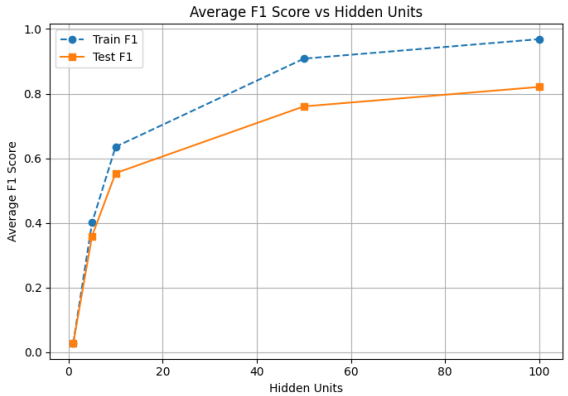

Figure 7: Average F1 Score vs Number of Hidden Units

Observations:

- The model performance (in terms of precision, recall, and F1) improves consistently as the number of hidden units increases.
- Smaller networks (e.g., 1 or 5 units) underfit the data, showing low F1 scores. Beyond 50 hidden units, both training and test scores become high, indicating effective learning, though the gap between train and test F1 (0.969 vs 0.821) for 100 units suggests mild overfitting.
- Overall, the model achieves the best balance between accuracy and generalization with around 50–100 hidden units.

2.3. Multiple Hidden Layers

In this section, we repeated the previous experiment using neural network architectures with multiple hidden layers to analyze the effect of network depth on model performance. The goal was to observe how increasing the number of layers impacts both the training and test metrics. We varied the network depth from one to four hidden layers while keeping other hyperparameters constant. The results are summarized in Table 9, and the corresponding trend is visualized in Figure 8.

Depth (Hidden Layers)	**Train Metrics**			**Test Metrics**		
	Precision	Recall	F1	Precision	Recall	F1
1 ([512])	0.997	0.997	0.997	0.884	0.884	0.884
2 ([512, 256])	0.999	0.999	0.999	0.893	0.893	0.893
3 ([512, 256, 128])	1.000	1.000	1.000	0.877	0.877	0.877
4 ([512, 256, 128, 64])	1.000	1.000	1.000	0.826	0.825	0.825

Table 9: Precision, Recall, and F1 Scores for Different Network Depths

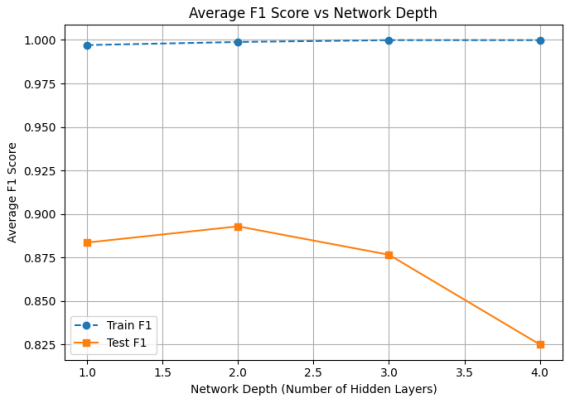

Figure 8: Effect of Network Depth on Test Performance (Precision, Recall, F1)

Observations:

- Training performance improves with increasing depth, reaching near-perfect scores for depth ≥ 3.
- Test performance initially improves slightly (peaking at depth 2), but then degrades as the model becomes deeper, suggesting overfitting.
- The drop in test F1 score from 0.893 (depth 2) to 0.825 (depth 4) indicates that additional layers hurt generalization on unseen data. A moderate-depth network (e.g., 2 layers) provides the best trade-off between complexity and generalization.

2.4. ReLU for activation

In this section, we repeated the experiment from part (c) by replacing the activation functions in the hidden layers with the ReLU activation unit, while keeping the final layer activation as softmax. Although ReLU is non-differentiable at $z = 0$, the use of sub-gradients allows us to handle this issue by assigning any value between the left and right derivatives for backpropagation. The objective is to observe the effect of ReLU on model training stability and generalization compared to the sigmoid-based networks. Table 10 summarizes the results for different network depths, and Figure 9 visualizes the trend in average F1-score versus depth.

Hidden Units	**Train Metrics**			**Test Metrics**		
	Precision	Recall	F1	Precision	Recall	F1
1	1.000	1.000	1.000	0.894	0.893	0.893
2	1.000	1.000	1.000	0.898	0.898	0.897
3	1.000	1.000	1.000	0.896	0.896	0.896
4	1.000	1.000	1.000	0.894	0.893	0.893

Table 10: Precision, Recall, and F1 Scores for Different Network Depths with ReLU Activation

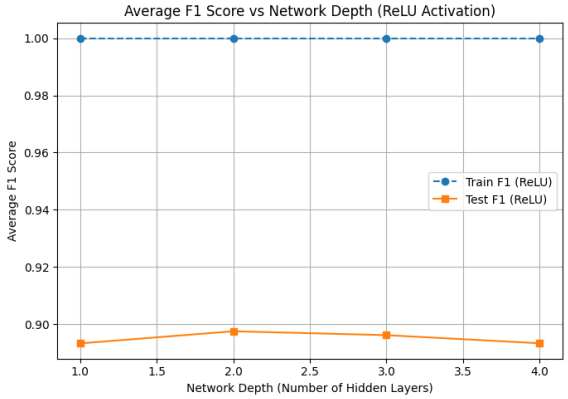

Figure 9: Average F1 Score vs Network Depth for ReLU Activation

To better understand the impact of activation choice, Table 11 compares the average test metrics between the Sigmoid (part c) and ReLU (part d) models.

Depth	**Sigmoid (Part c)**			**ReLU (Part d)**		
	Precision	Recall	F1	Precision	Recall	F1
1	0.884	0.884	0.884	0.894	0.893	0.893
2	0.893	0.893	0.893	0.898	0.898	0.897
3	0.877	0.877	0.877	0.896	0.896	0.896
4	0.825	0.825	0.825	0.894	0.893	0.893

Table 11: Comparison of Test Set Metrics between Sigmoid and ReLU Activations

Observations: From the results, it is evident that models using ReLU activation consistently achieve higher test precision, recall, and F1-scores across all depths compared to those using the sigmoid activation. ReLU networks also demonstrate faster convergence and better generalization, likely due to the absence of vanishing gradient issues present in sigmoid units. While both activations achieve perfect training performance, ReLU provides improved test performance, particularly for deeper architectures, suggesting that ReLU is more suitable for deeper networks.

2.5. MLPClassifier from scikit-learn

In this section, we implemented the same neural network architectures as in Part (d) using the `MLPClassifier` from the `scikit-learn` library. The models were trained with the `relu` activation function in the hidden layers, the `softmax` activation in the output layer (via cross-entropy loss), and the `sgd` solver. All other parameters were kept the same as before. Table 12 summarizes the precision, recall, and F1-score for both training and test sets across varying network depths. The trend of average F1-score with respect to network depth is illustrated in Figure 10.

Depth	**ReLU (Part d)**			**MLPClassifier (Part e)**		
	Precision	Recall	F1	Precision	Recall	F1
1	0.894	0.893	0.893	0.874	0.873	0.873
2	0.898	0.898	0.897	0.882	0.881	0.881
3	0.896	0.896	0.896	0.882	0.881	0.881
4	0.894	0.893	0.893	0.880	0.877	0.877

Table 12: Comparison of Test Set Metrics between ReLU Model (Part d) and MLPClassifier using ReLU

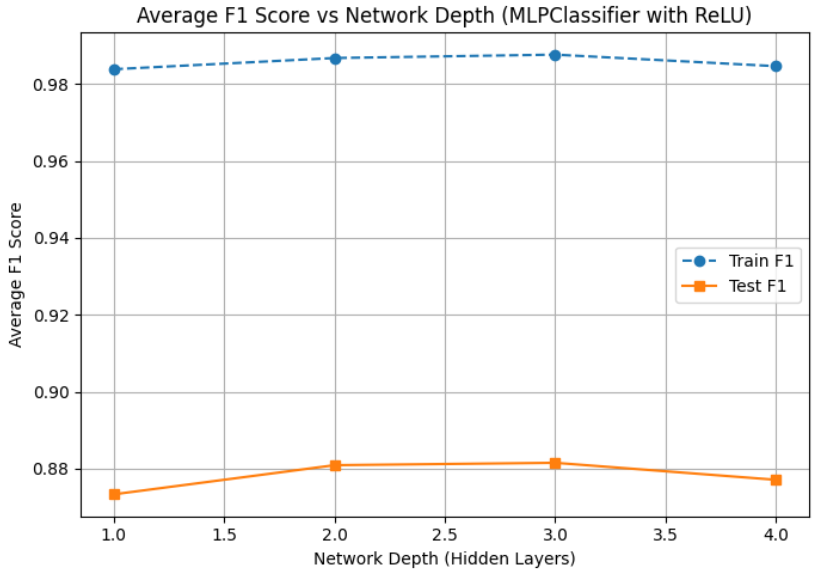

Figure 10: Average test F1-score vs. network depth for MLPClassifier with ReLU activation.

Observations: The results indicate that using the `MLPClassifier` with ReLU activation achieves performance comparable to the custom neural network in Part (d). Both models show a consistent test F1-score around 0.88 for deeper architectures. While the training metrics remain near perfect due to the model's capacity, test performance does not significantly improve beyond two hidden layers, suggesting that increasing depth beyond this point offers limited generalization benefit.

2.6. Transfer Learning

Transfer learning is a technique that leverages knowledge learned from one task to improve performance on a related task. Instead of initializing the model with random weights, a pre-trained model (in this case, trained on the *Consonant* subset with 36 classes) is used as a starting point for a new task (here, classifying the *Digits* subset with 10 classes). The intuition is that the early layers of the network have already learned generalizable low-level features such as edges and strokes that are useful across both datasets. Thus, initializing the network with these learned weights provides a better starting point than random initialization, leading to faster convergence and improved performance.

In this experiment, a 4-hidden-layer MLP with ReLU activations and hidden dimensions [512, 256, 128, 64] was trained on the digits dataset in two settings:

- **Training from Scratch:** The model was trained for 20 epochs with randomly initialized weights.

- **Transfer Learning (Fine-tuning):** The model initialized with weights from the consonant-trained network (except for the final output layer, which was reinitialized to have 10 neurons corresponding to the 10 digits) and was fine-tuned on the digits dataset for 20 epochs.

The final results after 20 epochs are as follows:

Model Type	Train F1	Test F1
From Scratch	0.943	0.926
Transfer Learning (Fine-tuned)	0.974	0.944

Table 13: Comparison of Train and Test F1-scores between models trained from scratch and fine-tuned using transfer learning on the Digits dataset.

The fine-tuned model clearly outperforms the one trained from scratch, both in terms of training and test F1-scores. This demonstrates the advantage of using a pre-trained model: it starts from a region in parameter space that already captures useful feature representations, allowing the model to adapt more effectively to the new but related digits classification task. The performance gap also suggests that the learned features from the consonant dataset generalize well to the digits subset.

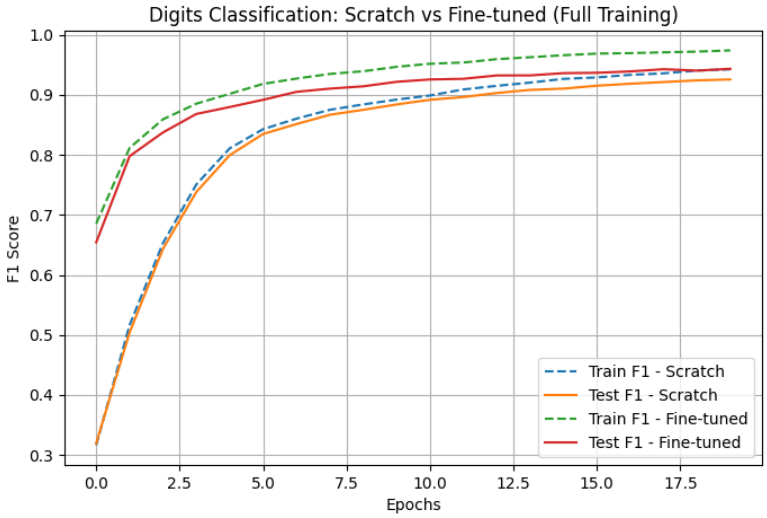

Figure 11: Train and Test F1-score comparison between Scratch and Transfer Learning models on the Digits dataset.